

Deliverable report

"Vittorio Polci": Mat: 257817, vittorio.polci@unitn.it, GitRepo: <https://github.com/vittorio01/GPU-Computing-2025-257817>

Abstract—This deliverable discusses about the implementation of different parallel solutions of the SpVM algorithm between CSR matrices and a general vector, with a final benchmark that compares all solution together.

Index Terms—Sparse Matrix, SpMV, CUDA, Parallelization, Storage Format

I. INTRODUCTION

The algorithm of sparse matrix-dense vector multiplication (SpMV) is an optimized version of the traditional matrix-vector multiplication designed to handle cases where the matrix is organized in a low density of non-zero entries. This technique is particularly useful in scenarios that involves very large sparse matrices because it allows to reduce the used space for storing that large structures and, at the same time, optimizes memory access patterns during computational steps.

Because the SpMV multiplication should handle matrices with millions or billions of not null elements, to obtain good results is necessary to use parallelization techniques on GPUs rather than sequential solution for CPUs.

II. PROBLEM STATEMENT

In the SpVM multiplication we consider a matrix coded in a sparse format (in this case the CSR format) and an input vector that has the same size of its columns number. Each elements of the resulting vector is computed by performing the dot products of each not null elements of the matrix with the input vector and accumulating them in the corresponding row position.

The main objective of this report is to propose a common solution for the SpMV for CUDA devices and propose more advanced optimizations techniques to improve the overall performances.

A. Storage Format

The CSR for sparse matrices is a storage format that involves three different arrays to represent the information:

- The data array, that contains all not null terms of the original matrix.
- The column array, where for each element in the data vector specifies its column coordinate.
- The row array, which contains the start and the end pointers for each row to the data and column arrays. these two pair of pointers indicates which not null terms in the data array are contained in that specific row.

The size of the data and column arrays depends on the number of not null elements and the size of the row array depends on the number of the rows in the original matrix.

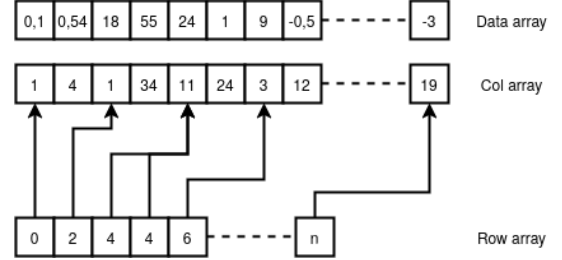


Fig. 1. Data organization in CSR structure

B. Parallelization

To improve the overall performance, the parallel implementations of the first algorithm are designed to be executed on Nvidia GPUs and programmed in CUDA. The CUDA language model permits to run a kernel on different concurrent threads, which are scheduled and executed by computational elements called CUDA cores.

To improve the performance and the organization of the parallel algorithms, CUDA architectures provides:

- *Warp organization*, where groups of 32 threads are able to synchronize and share data using predefined functions.
- *Block organization*, where more warps can synchronize themselves using atomic operations, barriers and have access to a dedicated shared memory.
- *Shared memory*, which is a fast but limited memory divided in more blocks and used to improve data reuse, optimize the global memory transfers and other purposes.
- *Read only cache*, which is a L1 read only cache that can be assigned by the programmer with the `__ldg()` function to improve data transfer for casual and repetitive accesses.

C. Warp reduction

In CUDA architectures there are predefined mechanisms to allow threads in the same warp to exchange data. The function `__shfl_down_sync()` permit to perform a reduction of arrays with maximum length of 32. For example, the function `reduceSum(acc)` uses five shuffle functions to accumulate all contribution given by threads in the same warp on the thread 0.

D. Ceiling division

The ceiling division is a system used to divide almost equally N elements across C equal size cells:

$$cellSize = \frac{N + C - 1}{C} \quad (1)$$

Algorithm 1 Warp reduction system

```
1: procedure REDUCESUM(acc)
2:    $acc = \_\_shfl\_down\_sync(0xffffffff, acc, 16)$ 
3:    $acc = \_\_shfl\_down\_sync(0xffffffff, acc, 8)$ 
4:    $acc = \_\_shfl\_down\_sync(0xffffffff, acc, 4)$ 
5:    $acc = \_\_shfl\_down\_sync(0xffffffff, acc, 2)$ 
6:    $acc = \_\_shfl\_down\_sync(0xffffffff, acc, 1)$ 
7: end procedure
```

From the obtained division, the *start point* and the *end point* for each cell are defined by:

$$start = cellSize * cId \quad (2)$$

$$end = \min(start + cellSize, N) \quad (3)$$

E. Binary Search

The binary search is an algorithm that finds the position of a certain element in a sorted array with a logarithmic complexity $O(\log(n))$. The proposed kernels use a modified version of the original algorithm to identify the position of the element nearest but lower or equal than the given value.

Algorithm 2 Adapted binary search

```
1: procedure BINARYSEARCH(array, size, element)
2:    $index = -1, left = 0, right = size$ 
3:   while  $left \leq right$  do
4:      $mid = (left + right) / 2$ 
5:     if  $array[mid] \leq value$  then
6:        $index = mid$ 
7:        $left = mid + 1$ 
8:     else
9:        $right = mid - 1$ 
10:    end if
11:  end while
12: end procedure
```

III. METHODOLOGY AND CONTRIBUTIONS

A. Dynamic algorithm (Parallel-2)

In the first dynamic algorithm, the matrix and the vector are processed using a row based algorithm that assigns a thread for each row:

- 1) The *rowArray* is splitted in equal subsection for different blocks using the ceiling division (1).
- 2) For each block a variable in the shared memory *pendingRow* is initialized to 0 and used to contain the next row number that is waiting to be processed.
- 3) Each thread uses the function *atomicAdd(pendingRow, 1)* to increment the value of *pendingRow* and, at the same time, read its content.
- 4) Using the value obtained, each thread computes the products and the accumulation of the entire row, saves the result in the output array and repeats step 3 and 4 until the block reach the end of its subsection.

Algorithm 3 Dynamic SpVM algorithm

Require: input, output and matrix structures.

```
1: procedure VMCSR MUL(output, matrix, input)
2:    $blockStart, blockEnd$   $\triangleright$  Ceiling division
3:    $pendingRow \leftarrow 0$   $\triangleright$  (only thread 0);
4:    $row = atomicAdd(pendingRow, 1) + blockStart$ 
5:   while  $row < blockEnd$  do
6:      $startRow, endRow$ 
7:      $\uparrow rowArray[row], rowArray[row + 1]$ 
8:      $acc = 0$ 
9:     while  $startRow < endRow$  do
10:       $col = colArray[startRow]$ 
11:       $acc +=$ 
12:       $dataArray[startRow] * input[col]$ 
13:       $startRow ++$ 
14:    end while
15:     $output[row] = acc$ 
16:     $row = atomicAdd(pendingRow, 1)$ 
17:     $+ blockStart$ 
18:  end while
19: end procedure
```

B. Optimized Dynamic algorithm (Parallel-5)

The dynamic algorithm can be improved further using a better organization of workload between threads. More performance can then be obtained by:

- Using the binary search in a preliminar kernel (6) to assign the rows to each block in base of the number of not-null elements of the matrix.
- Instead a single thread, *Assigning a warp for each row*. In this way, the threads in the warp process 32 partials from very long rows that can per reduced using the *reduceSum(1)*.
- Assigning the read only cache on the input vector to improve the speed by leveraging on the data reusage.

C. Shared cooperative algorithm (Parallel-6)

The algorithm *parallel-5* shows poor performances when considering heavily unbalanced matrices. Another approach that tries mitigate those cases divides the shared memory as a buffer where the workload is divided by threads in the same block:

- *sharedBuffer*, a space of size $B = blockSize.x * N$ used as buffer to contain the result of the multiplication between elements of the matrix and the input vector (each thread operates at max on N elements).
- *loadedRows*, a space of $R = blockSize.x$ cells used as cache for parts of the CSR row vector (each threads loads one pointer).
- *rowsBuffer*, a space of $R - 1$ cells used for accumulating partials for a small amount of rows (almost each thread maintains one cell).

Using these three different shared spaces, each block performs these steps:

Algorithm 4 Optimized dynamic algorithm (Parallel-5)

Require: matrix structure, block division array, input vector and output vector structures.

```

1: procedure VMCSRMUL(output, matrix, input,
   blockDivision)
2:   blockStart, blockEnd  $\triangleright$  binary search kernel
3:   pendingRow  $\leftarrow$  0  $\triangleright$  (only thread 0);
4:   row = RowElection
5:   while row < blockEnd do
6:     startRow, endRow
7:      $\uparrow$  rowArray[row], rowArray[row + 1]
8:     acc = 0
9:     while startRow < endRow do
10:      idx = min(startRow + laneId, endRow)
11:      col = colArray[idx]
12:      acc+ = fmaf(
13:        dataArray[startRow], __ldg(input[col])
14:        *(startRow + laneId < endRow))
15:      startRow ++
16:    end while
17:    acc = ReduceSum
18:    if laneId == 0 then
19:      output.dataArray[row] = acc
20:    end if
21:    row = RowElection
22:  end while
23: end procedure

```

Algorithm 5 Row election System

```

1: procedure ROWELECTION
2:   if laneId == 0 then
3:     row = atomicAdd(pendingRow, 1)
4:     +blockStart
5:   end if
6:   __shfl_sync(0xffffffff, row, 0)
7: end procedure

```

- 1) The entire block cooperates to *prepare the shared memory* for processing $R-1$ rows by initializing with zeroes the *rowsBuffer* space and loading the necessary CSR pointers from the row array in the *loadedRows* space.
- 2) The threads *load a block of elements* (maximum B elements) from the matrix and the input vector in a fully coalesced way, perform the multiplications and store the result in *sharedBuffer*.
- 3) The loaded elements *are divided then in equal strides* using the ceiling division (1) where each thread performs a binary search on *loadedRows* space to identify the correspondent row index of the first elements of their strides. Then the strides are reduced in partials and accumulated in the *rowsBuffer* using *atomicAdd()* in base of the correct row indexes.
- 4) The step 2 and 3 are performed until all necessary elements for the considered rows in *loadedRows* are

Algorithm 6 Preliminar kernel

Require: matrix structure, block division array initialized on global memory.

```

1: procedure VMCSRDIV(matrix, blockDivision)
2:   blockStart, blockEnd  $\triangleright$  Ceiling on notNull array
3:   if blockIdx.x == 0 then
4:     blockDivision[0] = 0
5:     blockDivision[blockDim.x] = rowSize
6:   else
7:     blockDivision[blockIdx.x]
8:      $\uparrow$  binarySearch(rowArray, rowSize, blockStart)
9:   end if
10: end procedure

```

$\triangleright$ Kernel launched for row division based on binary search. The kernel should be launched with same number of block of the main kernel with size equal to 1 thread

processed.

- 5) Each threads stores one result from the *rowsBuffer* to the output vector in the global memory.
- 6) All steps from 1 to 5 are repeated until all the rows of the matrix are processed.

The procedure 7 shows a heavily simplified representation of the shared cooperative algorithm.

IV. SYSTEM DESCRIPTION AND EXPERIMENTAL SET-UP

A. Dataset description

A dataset composed by 6 matrices is used for testing and benchmark all four algorithms. These matrices contains a number of not null elements between 1 and 1,5 million and they have different organization and dispersion that challenges the algorithms in different ways: The used dataset is obtained from the SuiteSparse Matrix Collection [1], a website that contains sample matrices useful for benchmarking algorithms.

Dataset	Dimension (R x C)	Non-zero entries	Type
dbir2 [2]	18,906 x 45,887	1,158,159	Unbalanced rows
ex11 [3]	16,614 x 16,614	1,096,948	Diagonally distributed (low density)
language [4]	399,130 x 199,130	1,216,334	Upper triangular
Linux_call_graph [5]	324,085 x 324,085	1,208,908	Dense matrix (worst case)
nemeth24 [6]	9,506 x 9,506	1,506,550	Diagonally distributed (best case)
twotone [7]	120,750 x 120,750	1,206,265	Diagonally distributed (moderate dispersion)

TABLE I
STRUCTURAL PATTERNS OF THE BENCHMARK SPARSE MATRICES.

B. System Description

The system used for the execution of the benchmarks uses CUDA 12.3.2 for compiling the source code, and is composed by a Intel(R) Xeon(R) Silver 4214 CPU and an Nvidia L40S GPU. The Nvidia L40S is based on Ada Lovelace architecture and has 142 Stream Multiprocessors with 96KB available shared memory per block, maximum memory bandwidth of 864GB/s and 91,61TFlops for FP32 operations.

Algorithm 7 Shared cooperative algorithm

Require: matrix structure, block division array, input vector and output vector structures.

```

1: procedure VMCSR_MUL(output, matrix, input,
   blockDivision)
2:   blockStart, blockEnd  $\triangleright$  (From binary search kernel).
3:   sharedBuffer, rowsBuffer, loadedRows
4:   for  $[0, (R-1)]$  rows in  $[blockStart, blockEnd]$  do
5:     if threadIdx.x < R then
6:       loadedRows[threadIdx.x]  $\leftarrow$  rowArray
7:       rowsBuffer[threadIdx.x] = 0
8:     end if
9:     for  $[0, B]$  in  $[loadedRows[0], loadedRows[R]]$  do
10:      for tId in  $[0, B]$ , coalesced do
11:        sharedBuffer[tId]  $\leftarrow$  dataArray[tId]
12:        * _ldg(input[colArray[tId]])
13:      end for
14:      tStart, tEnd  $\leftarrow$  Ceiling on B
15:      row  $\leftarrow$  binarySearch(loadedRows, B, tStart)
16:      for i in  $[tStart, tEnd]$  do
17:        row  $\leftarrow$  row adjusting
18:        atomicAdd(rowsBuffer[row], i)
19:      end for
20:    end for
21:    if threadIdx.x < (R - 1) then
22:      output  $\leftarrow$  rowsBuffer[threadIdx.x]
23:    end if
24:  end for
25: end procedure

```

C. Experimental Set-up

The performance is measured using the value of GFlops obtained by the execution time elapsed measured with *cudaEventRecord* functions for CUDA algorithms and considering also the DRAM throughput, Compute utilization and Shared memory efficiency.

The results obtained by the three kernels are then compared with the library cuSPARSE [8], the official library supported by Nvidia.

D. Block size estimation

The parameters used for launching the kernels are calculated empirically in base of the amount of shared memory available per block, the number of cuda cores per SM and the size of the input matrix. In particular:

- We choose the number of blocks equal to the number of not null elements of the input matrix divided by a constant obtained empirically of around 3328 (*notNull/NotNullPerBlock*).
- In the kernel 7, the dimension of the shared memory is related on the number of elements that each thread should load and the block dimension (*shared* = (*elementsPerThread* + 2) * *blockDim* * *sizeof(float*)).

- We can obtain the number of threads per block by considering *N* block active at the same time in a Stream Multiprocessor, which is limited to the shared memory for the kernel *parallel-6*. In this case we consider the number of threads equal to $sharedPerSM(Kb) \div [N * (elementsPerThread + 2) * sizeof(float)]$. From various benchmarks, the best performances are obtained when considering *N* = 2 and *elementsPerThread* = 14. For the kernels *parallel-2* and *parallel-5* the number of threads can also be fixed at a value of 512 or 256 to obtain stable performances.

V. EXPERIMENTAL BENCHMARKS

The following tables shows the values of GFLOPs/s (II) and effective bandwidth (??) obtained from benchmarking on 256 blocks of 512 using float values. The figure 2 gives also a visualization of table II.

Matrix	Parallel 1	Parallel 2	Parallel 3	Sequential
dbir2 [2]	152,93	142,78	34,8	1,69
ex11 [3]	71,44	71,74	21,02	0,38
language [4]	31	82,4	1,91	0,32
Linux_call_graph [5]	0,85	1,72	2,51	0,25
nemeth24 [6]	43,4	50	16,81	0,37
twotone [7]	44,42	51,84	4,7	0,42

TABLE II
GFLOPS/S COMPARISON BETWEEN PARALLEL AND SEQUENTIAL IMPLEMENTATIONS

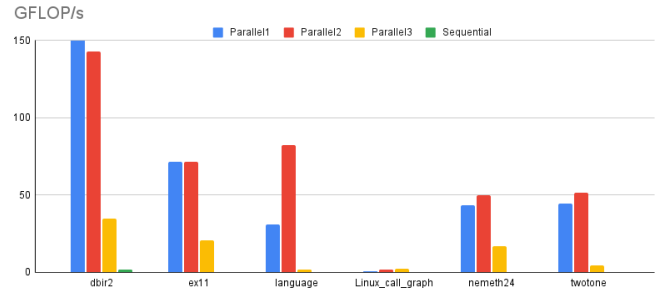


Fig. 2. Performance differences between the four algorithms

VI. RESULTS AND CONCLUSION**REFERENCES**

- [1] U. of Florida. Suitesparse matrix collection. [Online]. Available: <https://sparse.tamu.edu/>
- [2] Meszaros. dbir2 matrix. [Online]. Available: <https://sparse.tamu.edu/Meszaros/dbir2>
- [3] FIDAP. ex11 matrix. [Online]. Available: <https://sparse.tamu.edu/FIDAP/ex11>
- [4] Tromble. language matrix. [Online]. Available: <https://sparse.tamu.edu/Tromble/language>
- [5] Sorensen. Linux call graph matrix. [Online]. Available: https://sparse.tamu.edu/Sorensen/Linux_call_graph
- [6] Nemeth. nemeth24 matrix. [Online]. Available: <https://sparse.tamu.edu/Nemeth/nemeth24>
- [7] ATandT. twotone matrix. [Online]. Available: <https://sparse.tamu.edu/ATandT/twotone>
- [8] NVIDIA. cusparse. [Online]. Available: <https://developer.nvidia.com/cusparse>